PYTHON · MUSIC · CODE

Getting Started with Python

A 2-hour coding workshop

```
import coffee
import curiosity
import patience

assert fear_of_coding == False, "You've got
this!"

fun_fact = "Python is named after Monty Python,
not the snake"
print(fun_fact)
print("Good morning, Walthamstow!")

# We'll be getting started shortly ...
current_date = "Saturday 11 April 2026"
current_time = "01:48:59.925"

presenters = [
    "Kapilan M",
    "Elisha Gretton",
    "Becks Simpson",
    "Mohini Thakkar",
]
```

How today works

Over the next **2 hours** you'll learn the core building blocks of Python — using songs, playlists, albums, and lyrics as your playground. Every section follows the same rhythm:

📖 Live example on screen → 💻 **Exercise — your turn** → Next section

0 · Setup

1 · Variables & Strings

2 · Lists

3 · Loops

4 · Conditionals

5 · Dictionaries

6 · Functions

7 · File I/O

Finished early? Scroll to the **Extension Challenge** at the bottom of your exercise file — or open the **Data Explorer** to browse 9,000+ real songs.

EXERCISE 0

Getting Set Up

Step 1 — Get the workshop folder

Go to **tinyurl.com/python-walthamstow-workshop** → click the green **Code** button → **Download ZIP**

Then unzip it:

Mac

Double-click the `.zip` — a folder appears next to it

Windows

Right-click the `.zip` → **Extract All...** → **Extract**

Move the folder somewhere easy to find, like your **Desktop**.

Step 2 — Run the setup script

🍎 Mac

1. Open **Terminal** — press `Cmd+Space`, type Terminal, press Enter
2. Type `bash` with a space (don't press Enter yet)
3. Open the `setup` folder → drag `setup.command` into the Terminal window
4. Press Enter

Windows

1. Open the `setup` folder in File Explorer
2. Click the **address bar** at the top → type `cmd` → press Enter
3. Type `setup.bat` and press Enter

If you see this, you're all good:

```
[ OK ] Python 3.12.3 (bundled)
[ OK ] Loaded 9237 songs.

*** ALL DONE — YOU'RE GOOD TO GO! ***
```

exercise_00_setup_check.py

Step 3 — Install & Open VS Code

VS Code is the program you'll write Python in.

If you don't have VS Code yet: go to **code.visualstudio.com** → download and install it

1. Open **VS Code**
2. **File → Open Folder...**
3. Find your workshop folder → select it → click **Open**
4. You'll see the `exercises` folder appear in the left panel

> **Mac:** if VS Code asks **"Do you trust the authors?"** → click **Yes, I trust the authors**

Step 4 — Run your first exercise

Open the terminal inside VS Code:

Press `Ctrl+` ` — the **backtick** key, top-left of the keyboard next to `1`

A panel opens at the bottom of VS Code. Type this and press Enter:

```
python3 exercises/exercise_00_setup_check.py
```

You should see:

```
Everything is set up correctly — you're ready to go!
```

Windows: if `python3` isn't recognised, try `python` instead

exercise_00_setup_check.py

EXERCISE 1

Variables & Strings

What is a variable?

A variable is a **labelled box** that stores a piece of information.

`song_title` → `"Blinding Lights"`

`artist` → `"The Weeknd"`

`year` → `2019`

Text goes inside quote marks: `"like this"`
Numbers don't need quotes: `2019`, `3.22`

exercise_01_variables_and_strings.py

Variables — code

variables.py

```python
song_title = "Blinding Lights"
artist     = "The Weeknd"
year       = 2019
duration   = 3.22   # minutes

print(song_title)   # Blinding Lights
print(artist)       # The Weeknd
print(year)         # 2019
```

exercise_01_variables_and_strings.py

f-strings

Slot variables directly into text — put `f` before the quote and `{}` around variable names.

```
song_title = "Blinding Lights"
```

```
artist = "The Weeknd"
```

```
year = 2019
```

```
f"Now Playing: {song_title} by {artist} ({year})"
```

output

press Evaluate to run…

▶ Evaluate

exercise_01_variables_and_strings.py

Your Task — Exercise 1

💻 `exercise_01_variables_and_strings.py`

1. Create variables for your favourite song:
   - `my_song_title`
   - `my_artist`
   - `my_year`
   - `my_duration`
2. Use an f-string to print:

```
Now Playing: <title> by <artist> | Released: <year> | Duration: <duration> mins
```

TIMER

03:00

Bossa Beyond

▶ ↻

✋ Stuck? Raise your hand and a volunteer will come to you!

⏳ Need more time? Don't worry, we can extend the timer — we are in no rush.

EXERCISE 2

Lists

What is a list?

A list stores **multiple items in order** — each item has an index starting at 0.

[0]	[1]	[2]	[3]
Blinding Lights	Levitating	Stay	Heat Waves

`playlist[0]`

the **first** item

`playlist[-1]`

the **last** item

`len(playlist)`

how many items

exercise_02_lists.py

Lists — code

lists.py

```
playlist = ["Blinding Lights", "Levitating", "Stay", "Heat Waves"]

print(playlist)        # the whole list
print(playlist[0])     # first item  → "Blinding Lights"
print(playlist[-1])    # last item   → "Heat Waves"
print(len(playlist))   # how many    → 4

playlist.append("As It Was")     # add to the end
playlist.remove("Stay")          # remove a specific song
playlist.insert(0, "Flowers")    # insert at position 0 (the start)
```

exercise_02_lists.py

Your Task — Exercise 2

exercise_02_lists.py

1. Create `my_playlist` with at least 5 song titles
2. Print the entire playlist
3. Print only the **third** song
4. Use `.append()` to add a new song to the end
5. Use `.remove()` to delete one song
6. Print the total number of songs using `len()`

TIMER

03:00

Bossa Beyond

✋ Stuck? Raise your hand and a volunteer will come to you!

⏳ Need more time? Don't worry, we can extend the timer — we are in no rush.

EXERCISE 3

Loops

What is a loop?

Instead of writing the same line for every song — let the loop do it.

```
[0]  Blinding Lights
[1]  Levitating
[2]  Stay
[3]  Heat Waves
```

output

press play to run the loop…

▶ play step → ↺ reset

exercise_03_loops.py

Loops — code

loops.py

```python
playlist = ["Blinding Lights", "Levitating", "Stay", "Heat Waves"]

for song in playlist:
    print(f"Now playing: {song}")
```

exercise_03_loops.py

range() — repeat a fixed number of times

`range(4)` generates the numbers **0, 1, 2, 3** — the loop body runs once for each.

`range(4)` produces:

i = 0	i = 1	i = 2	i = 3

output

press play…

▶ play step → ↺ reset

exercise_03_loops.py

range() — the code

range.py

```python
chorus = "We will, we will rock you!"

for i in range(4):
    print(chorus)

# Output:
# We will, we will rock you!
# We will, we will rock you!
# We will, we will rock you!
# We will, we will rock you!
```

exercise_03_loops.py

enumerate() — loop with a counter

Instead of just the item, you also get its **index** — great for numbered lists.

1 Blinding Lights

2 Levitating

3 Stay

4 Heat Waves

output

press play…

▶ play step → ↺ reset

exercise_03_loops.py

enumerate() — the code

enumerate.py

```python
playlist = ["Blinding Lights", "Levitating", "Stay", "Heat Waves"]

for i, song in enumerate(playlist, start=1):
    print(f"{i}. {song}")

# Output:
# 1. Blinding Lights
# 2. Levitating
# 3. Stay
# 4. Heat Waves
```

Your Task — Exercise 3

exercise_03_loops.py

1. Create a list of 5 songs. Loop through and print each one with its track number using `enumerate()`
2. Pick a short lyric. Use `range()` to print it four times
3. Loop through and print every title in **UPPERCASE** using `.upper()`

TIMER

03:00

Bossa Beyond

✋ Stuck? Raise your hand and a volunteer will come to you!

⏳ Need more time? Don't worry, we can extend the timer — we are in no rush.

EXERCISE 4

Conditionals

What is a conditional?

Your code takes a **different path** depending on whether something is true.

song_bpm = **140**

Blinding Lights **171** | Levitating **103** | Stay **90** | Heat Waves **81**

```
if    140 > 150   false ✗   →  "High energy!"
elif  140 > 100   true ✓    →  "Mid-tempo"
else  (no condition)  skipped  →  "Slow and relaxed"
```

exercise_04_conditionals.py

Conditionals — code

conditionals.py

```
song_bpm = 140

if song_bpm > 150:
    print("High energy — great for a workout!")
elif song_bpm > 100:
    print("Mid-tempo — good for studying.")
else:
    print("Slow and relaxed — wind-down music.")
```

exercise_04_conditionals.py

Conditionals inside a loop

conditionals_loop.py

```python
titles = ["Blinding Lights", "Savage",    "Levitating"]
bpms   = [171,               134,         103]

for i, title in enumerate(titles):
    bpm = bpms[i]
    if bpm > 130:
        print(f"{title} — High energy")
    else:
        print(f"{title} — Mid-tempo or slower")
```

exercise_04_conditionals.py

Loop + conditional — step by step

For each song the loop picks up its BPM, then the `if` decides which branch runs.

titles	bpms
Blinding Lights	171
Savage	134
Levitating	103

conditional

```
if    bpm > 130
    → "High energy"

else  (no condition)
    → "Mid-tempo or slower"
```

output

press play...

▶ play step → ↺ reset

exercise_04_conditionals.py

Your Task — Exercise 4

exercise_04_conditionals.py

1. Create two lists: song titles and BPM values (at least 5 songs, BPM range 60–200)
2. Loop through and classify each song:
   - **High energy** → BPM > 130
   - **Mid-tempo** → BPM 90–130
   - **Slow** → BPM < 90
3. Add a skip condition: if BPM > 180, print `"Skipping <title> — too fast!"`

TIMER

03:00

Bossa Beyond

✋ Stuck? Raise your hand and a volunteer will come to you!

⏳ Need more time? Don't worry, we can extend the timer — we are in no rush.

EXERCISE 5

Dictionaries

What is a dictionary?

Group related data under named **keys** — like a labelled form.

`"title"`	`"Future Nostalgia"`
`"artist"`	`"Dua Lipa"`
`"year"`	`2020`
`"tracks"`	`{ 1: "Future Nostalgia", 2: "Don't Start Now", … }`

Access any value with `album["key"]`

exercise_05_dictionaries.py

Dictionaries — code

dictionaries.py

```python
album = {
    "title":  "Future Nostalgia",
    "artist": "Dua Lipa",
    "year":   2020,
    "tracks": {
        1: "Future Nostalgia",
        2: "Don't Start Now",
        3: "Cool",
    }
}

print(album["title"])       # Future Nostalgia
print(album["tracks"][2])   # Don't Start Now

for track_num, track_title in album["tracks"].items():
    print(f"Track {track_num}: {track_title}")
```

exercise_05_dictionaries.py

Your Task — Exercise 5

exercise_05_dictionaries.py

1. Create `my_album` with keys:
   - `title`, `artist`, `year`, `genre`
   - `tracks` — a dict mapping track numbers → song titles
2. Print the album title and artist on one line using an f-string
3. Loop through the tracks and print a numbered listing
4. Add a new track to the dictionary after you've created it

TIMER

03:00

Bossa Beyond

✋ Stuck? Raise your hand and a volunteer will come to you!

⏳ Need more time? Don't worry, we can extend the timer — we are in no rush.

EXERCISE 6

Functions

What is a function?

Define code **once**, call it as many times as you like — with different inputs each time.

```
title="Blinding Lights"
artist="The Weeknd"
```

↓

now_playing()

↓

```
▶ Blinding Lights — The Weeknd
```

```
title="Levitating"
artist="Dua Lipa"
```

↓

now_playing()

↓

```
▶ Levitating — Dua Lipa
```

exercise_06_functions.py

Why use functions?

✗ without a function

```
1  print("▶  Blinding Lights — The Weeknd")
2  print("▶  Levitating — Dua Lipa")
3  print("▶  Heat Waves — Glass Animals")
4  # Change the format? Edit every single line
```

✓ now_playing()

```
1  def now_playing(title, artist):
2      print(f"▶  {title} — {artist}")
3
4  now_playing("Blinding Lights", "The Weeknd"
5  now_playing("Levitating",      "Dua Lipa")
6  now_playing("Heat Waves",      "Glass Anima
7  # Change the format? Edit one line.
```

exercise_06_functions.py

Functions that return a value

classify_bpm()

```python
def classify_bpm(bpm):
    if bpm > 130:
        return "High energy"
    elif bpm > 90:
        return "Mid-tempo"
    else:
        return "Slow"

print(classify_bpm(171))   # High energy
print(classify_bpm(96))    # Mid-tempo
print(classify_bpm(68))    # Slow
```

exercise_06_functions.py

Your Task — Exercise 6

exercise_06_functions.py

1. Write `describe_song(title, artist, year)` — prints one formatted line. Call it 3 times.

2. Write `longest_title(songs)` — prints the longest title and its character count:

```
Longest title: Someone Like You (16 characters)
```

TIMER

03:00

Bossa Beyond

✋ Stuck? Raise your hand and a volunteer will come to you!

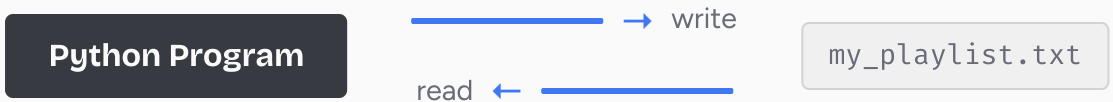
⏳ Need more time? Don't worry, we can extend the timer — we are in no rush.

EXERCISE 7

File I/O

Why save to a file?

So far, data disappears when the program stops. A file keeps it around.



`"w"` write · `"r"` read · `"a"` append

exercise_07_file_io.py

Writing to a file

write_playlist.py

```python
playlist = [
    "Blinding Lights — The Weeknd",
    "Levitating — Dua Lipa",
    "Heat Waves — Glass Animals",
]

with open("my_playlist.txt", "w") as f:
    for song in playlist:
        f.write(song + "\n")   # \n = new line

print("Playlist saved!")
```

exercise_07_file_io.py

Reading a file back

read_playlist.py

```python
with open("my_playlist.txt", "r") as f:
    lines = f.readlines()

print(f"Loaded {len(lines)} songs:")
for i, song in enumerate(lines, start=1):
    print(f"{i}. {song.strip()}")   # .strip() removes \n
```

exercise_07_file_io.py

Your Task — Exercise 7

exercise_07_file_io.py

1. Create a list of at least 5 songs in `"Title – Artist"` format. Write them to a file called `my_playlist.txt`.

2. Read `my_playlist.txt` back and print a numbered track listing. Remember to use `.strip()` so the `\n` doesn't show up.

TIMER

03:00

Bossa Beyond

▶ ↻

✋ Stuck? Raise your hand and a volunteer will come to you!

⏳ Need more time? Don't worry, we can extend the timer — we are in no rush.

You’ve made it through all 7 exercises!

You now know the core building blocks of Python:

- variables
- lists
- loops
- conditionals
- dictionaries
- functions
- file I/O

Want to explore 9,000+ real songs?

Open the Data Explorer from the `setup` folder:

Platform	File
Mac	`setup/explore.command`
Windows	`setup/explore.bat`

Search by song, artist, or genre — then use what you find in your exercises.